

Aula 40: Monitoramento e Observabilidade - Teoria e Prática

Duração: 4 horas (2h teoria + 2h prática)

Parte Teórica (2h) - Conceitos Essenciais

1. Pilares da Observabilidade

Pilar	Ferramentas	O que monitora?
Métricas	Prometheus, Grafana	CPU, memória, taxa de mensagens
Logs	ELK Stack, Loki	Erros, eventos de sistema
Tracing	Jaeger, Zipkin	Fluxo de requisições distribuídas

2. Por que Monitorar Aplicações Reativas?

- **Backpressure:** Identificar gargalos no fluxo de dados.
- **Conexões WebSocket:** Monitorar sessões ativas e tempo de vida.
- **Event Loop:** Verificar bloqueios em operações I/O.

3. Métricas Chave para o Nosso Chat

- **Backend:**
 - Mensagens processadas/segundo.
 - Tempo médio de resposta WebSocket.
- **Frontend:**
 - Latência da API.
 - Erros de conexão WebSocket.

4. Arquitetura do Sistema de Monitoramento



```
B --> C (Grafana)
A --> |Logs| D (Loki)
D --> C
E[Frontend] --> |Erros| F (Sentry)
```

Parte Prática (2h) - Implementando Monitoramento

1. Configurando Prometheus + Grafana (1h)

Passo 1: Adicione Actuator ao Spring Boot

gradle

```
// build.gradle
implementation 'org.springframework.boot:spring-boot-starter-actuator'
implementation 'io.micrometer:micrometer-registry-prometheus'
```

Passo 2: Configure o endpoint de métricas

yaml

```
# application.yml
management:
  endpoints:
    web:
      exposure:
        include: health, prometheus, metrics
  metrics:
    tags:
      application: reactive-chat
```

Passo 3: Docker Compose para Monitoramento

yaml

```
# docker-compose-monitoring.yml
services:
  prometheus:
    image: prom/prometheus
    ports: ["9090:9090"]
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml

  grafana:
    image: grafana/grafana
```

```
ports: ["3000:3000"]
depends_on: ["prometheus"]
```

Arquivo: prometheus.yml

yaml

```
scrape_configs:
  - job_name: 'spring'
    metrics_path: '/actuator/prometheus'
    static_configs:
      - targets: ['backend:8080']
```

2. Dashboard no Grafana (1h)

Importe este JSON para o Grafana:

json

```
{
  "title": "Chat Metrics",
  "panels": [
    {
      "title": "Mensagens por Segundo",
      "type": "graph",
      "targets": [{
        "expr": "rate(messages_processed_total[1m])",
        "legendFormat": "{{instance}}"
      }]
    },
    {
      "title": "Conexões WebSocket Ativas",
      "type": "stat",
      "targets": [{
        "expr": "websocket_sessions_active",
        "legendFormat": "Sessões"
      }]
    }
  ]
}
```

Atividade:

1. Acesse `http://localhost:3000` (Grafana).
2. Adicione uma nova fonte de dados (Prometheus em `http://prometheus:9090`).
3. Importe o JSON acima e personalize o dashboard.

Atividade Avançada

Desafio: Implemente log estruturado com Loki:

1. Backend: Configure logs em JSON:

```
yaml

logging:
  pattern:
    console: '{"timestamp":"%d{ISO8601}","level":"%level","message":"%m"}'
```

2. Frontend: Capture erros com Sentry:

```
typescript

import * as Sentry from '@sentry/vue';

Sentry.init({
  dsn: 'SUA_DSN',
  integrations: [new Sentry.BrowserTracing()],
});
```

Solução:

```
yaml

# docker-compose-monitoring.yml (adicione)
loki:
  image: grafana/loki
  ports: ["3100:3100"]

promtail:
  image: grafana/promtail
  volumes:
    - /var/log:/var/log
  command: ["-config.file=/etc/promtail/config.yml"]
```

Recursos Adicionais

- Grafana Dashboards Prontos: <https://grafana.com/grafana/dashboards/>
- Spring Boot Metrics: <https://docs.spring.io/spring-boot/docs/current/reference/html/actuator.html>

Próximos Passos (Opcional):

- Configure alertas no Grafana para métricas críticas.
- Adicione tracing distribuído com Jaeger.

Projeto Finalizado com Observabilidade! 🚀